

APIs & Communication Contracts

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles

An **API (Application Programming Interface)** is a contract: it defines exactly how one piece of software can ask another for something, without either side needing to know the other's internal implementation. You write these contracts constantly in Laravel/Node — every route file is a small API contract. HLD asks you to think about that contract at the scale of services talking to services, not just a browser talking to your app.

Synchronous vs. Asynchronous communication

Synchronous communication means the caller sends a request and blocks, waiting for a response before continuing (a normal REST call). **Asynchronous** communication means the caller sends a message and moves on immediately, without waiting for the work to finish (e.g., pushing a job onto a queue). Almost every HLD interview eventually asks: "should this call be sync or async?" — and the answer depends on whether the caller actually needs the result immediately.

Backend engineer analogy

Dispatching a Laravel Job onto a queue instead of running it inline in the controller is exactly the sync-vs-async decision, just inside one app instead of between services. Sending a welcome email synchronously during signup makes the user wait for an SMTP round-trip they don't care about — queueing it is the async, correct choice. The same reasoning scales up to inter-service calls.

REST vs. gRPC

REST represents resources as URLs and uses standard HTTP verbs and JSON payloads. It's human-readable, universally supported, and the default choice for public-facing and browser-facing APIs.

gRPC is a binary RPC (Remote Procedure Call) framework built on HTTP/2, using Protocol Buffers (protobuf) instead of JSON. It's significantly faster and smaller on the wire than REST/JSON, supports streaming natively, and enforces a strict, typed contract between services. The trade-off: it's not human-readable in a browser, and tooling/debugging is heavier. gRPC is the common choice for *internal* service-to-service calls at companies like Uber, Google, and Netflix, while REST usually stays at the public edge.

One more foundational idea worth naming here, even though Phase 2 covers it in full: whenever you make a call that might be retried — and in distributed systems, almost any call might be retried after a timeout — you need to think about **idempotency**: can this operation be safely repeated without causing a duplicate side effect (like charging a customer twice)? A well-designed API contract makes this explicit, often via an idempotency key the caller supplies, so the server can recognize "I've already done this" and simply return the original result instead of repeating the action.

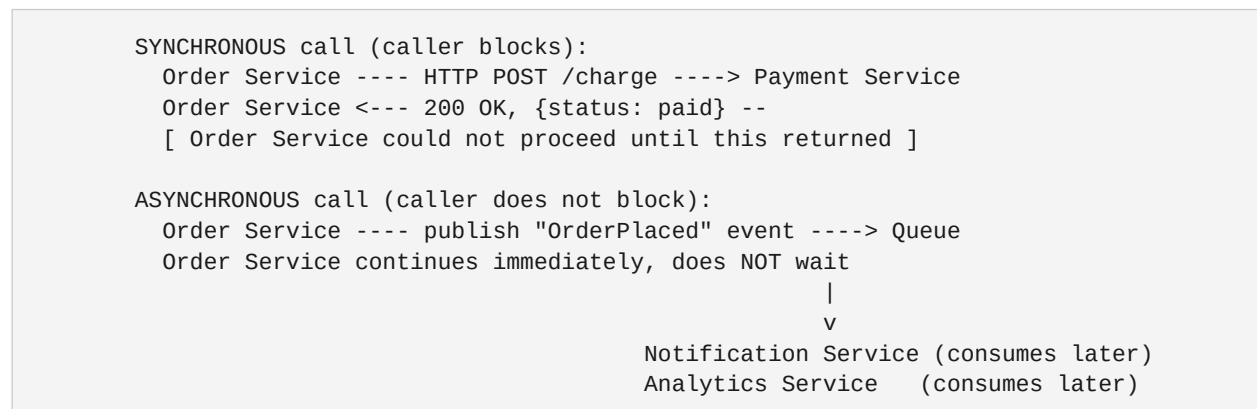
2. Real-World Example / Case Study

Razorpay's public payment API is REST — merchants integrate over standard HTTPS/JSON because it needs to be usable by every programming language and every developer skill level, with clear documentation. Internally, however, high-throughput microservices at large fintechs (payment authorization talking to fraud-check talking to ledger service) often use gRPC between themselves, because the calls happen millions of times a day and the extra performance and strict typing genuinely matter at that volume.

Swiggy's order flow illustrates sync vs. async well: the checkout API call (create the order, charge the payment) is synchronous — the user is waiting and needs a real answer. But triggering the restaurant's kitchen printer, sending the order-confirmation push notification, and updating analytics dashboards are all fired asynchronously via a queue, because the checkout response shouldn't wait on any of them.

A third example: PhonePe's merchant settlement API exposes both a synchronous "check status" endpoint (a merchant's system asks "what's the current state of this payout?" and gets an immediate answer) and an asynchronous webhook (PhonePe proactively calls the merchant's server the moment the payout completes). Offering both is a common, deliberate pattern — sync for on-demand checks, async for timely notification — rather than forcing integrators to poll constantly for a status that might not change for hours.

3. Diagram



One more layer worth naming: API contracts also need a versioning strategy, since a contract that never changes is unrealistic for any product evolving over years. Common approaches include putting a version in the URL (*/v1/orders*, */v2/orders*), a custom header, or content negotiation. The core discipline is the same regardless of mechanism: never make a breaking change to a contract other services or clients already depend on — add new fields/endpoints instead, and deprecate old ones on a clear timeline.

4. Trade-offs Table

Aspect	REST (JSON over HTTP)	gRPC (Protobuf over HTTP/2)
Readability	Human-readable, browser-debuggable	Binary, needs tooling to inspect
Performance	Slower, larger payloads	Faster, smaller payloads, native streaming

Aspect	REST (JSON over HTTP)	gRPC (Protobuf over HTTP/2)
Typing	Loose unless enforced separately (OpenAPI)	Strict, code-generated from .proto contract
Best for	Public APIs, browser clients, simplicity	Internal service-to-service, high throughput
Adoption cost	Near zero, universally supported	Requires codegen tooling, steeper learning curve

5. Common Interview Questions

Q1: "When would you choose gRPC over REST?"

Model approach: State that gRPC shines for internal, high-volume, latency-sensitive service-to-service calls with a stable contract between teams you control, while REST remains better for public APIs or anything a browser needs to call directly. Mention gRPC's native support for streaming as a differentiator.

Q2: "How do you decide if a call should be synchronous or asynchronous?"

Model approach: Ask "does the caller need the result to proceed, and does the user expect an immediate answer?" If yes to both, keep it synchronous. If the work can happen in the background without blocking the user-facing response, make it asynchronous via a queue — and mention the trade-off: async adds eventual-consistency complexity (Chapter 8, Phase 2's messaging chapters go deeper here).

Q3: "How would you design an API so that a client accidentally sending the same 'place order' request twice doesn't create two orders?"

Model approach: Require the client to generate and send a unique idempotency key with the request; the server checks whether it has already processed that key before creating a new order, and if so, returns the original response instead of creating a duplicate. Mention this key must be stored durably alongside the result, at least for a reasonable retry window.

6. Practice Exercises

- 1 For an e-commerce checkout, list 3 calls that must be synchronous and 3 that should be asynchronous, with a one-line reason each.
- 2 Explain, without notes, the core difference between REST and gRPC to a non-technical friend using an analogy.
- 3 Sketch an API contract (just the endpoint, method, and JSON shape) for "place an order" as if you were designing it today.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"REST vs gRPC in Microservices Which One Should You Use?"	Practical, decision-focused comparison for microservices.

Language	Video & Channel	Why it helps
English	"gRPC vs REST API – Speed, Streaming & Scalability Explained"	Focuses on the performance trade-offs interviewers probe.
Hindi	"REST vs gRPC in Microservices" (Hinglish explanation available on the same video)	Widely referenced comparison, accessible to Hindi-English mixed learners.
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp	Covers API design choices as part of a broader Hindi system design roadmap.

8. Key Takeaways

- APIs are contracts between software components, independent of internal implementation.
- Synchronous calls block the caller for a result; asynchronous calls let the caller move on immediately.
- Choose sync when the caller genuinely needs the result now; choose async (queue-based) otherwise.
- REST (JSON/HTTP) is the default for public and browser-facing APIs — simple, universal.
- gRPC (Protobuf/HTTP2) is preferred for internal, high-throughput, latency-sensitive service calls.
- Real production systems mix both: REST at the edge, gRPC or async messaging internally.